

SWE40006 – Software Deployment and Evolution

Deployment Portfolio Task 1

Daniel Johansson Tan (105075074)

Semester 2

Submission Date: 21 August 2026

Target Level: 1.4 (HD)

GitHub: <https://github.com/chentartist/Software-deployment-and-evolution>

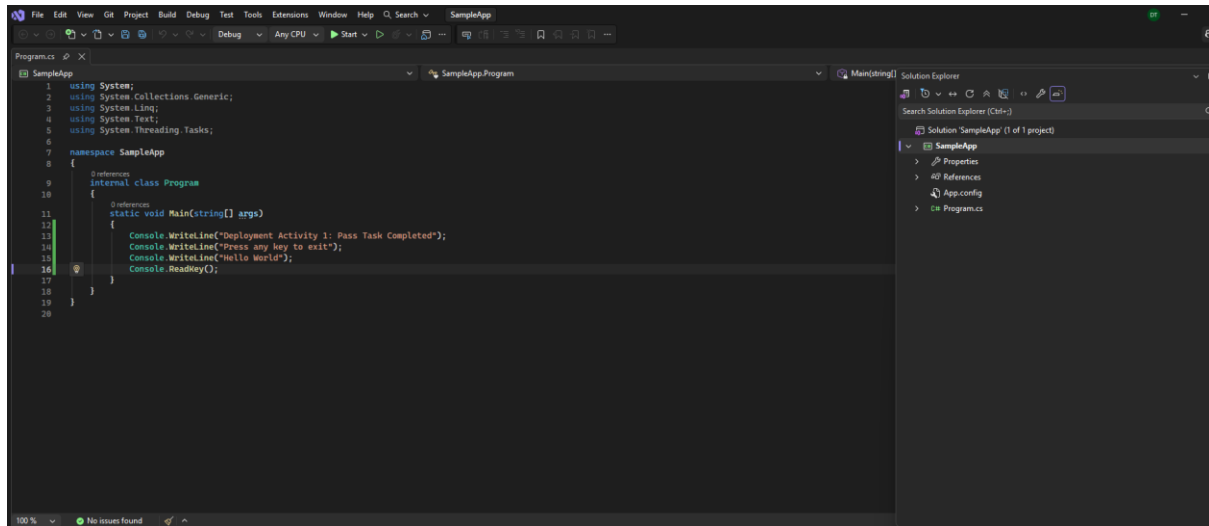
Table of Contents

1.1	Sample App	3
1.2	Simple Text Processor	6
1.3	Adding DLLs	10
1.4	GitHub Release + WinGet.....	14

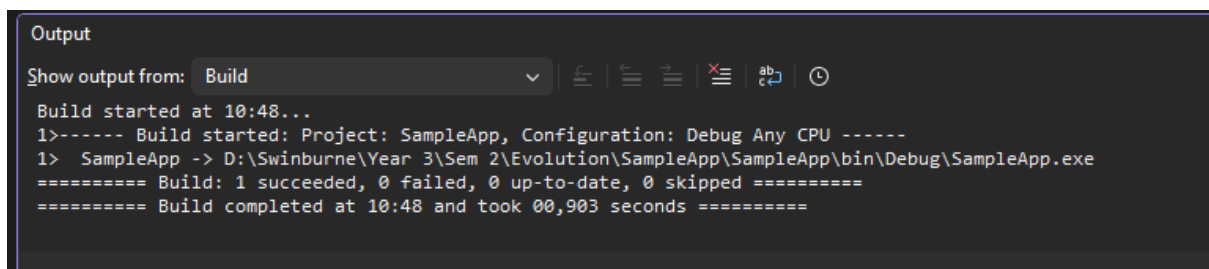
1.1 Sample App

For the first task I followed the instruction given in the assignment to download all the necessary software, like visual studio, Wix, etc; and building a simple Sample app to print out some sentences like “Hello World”

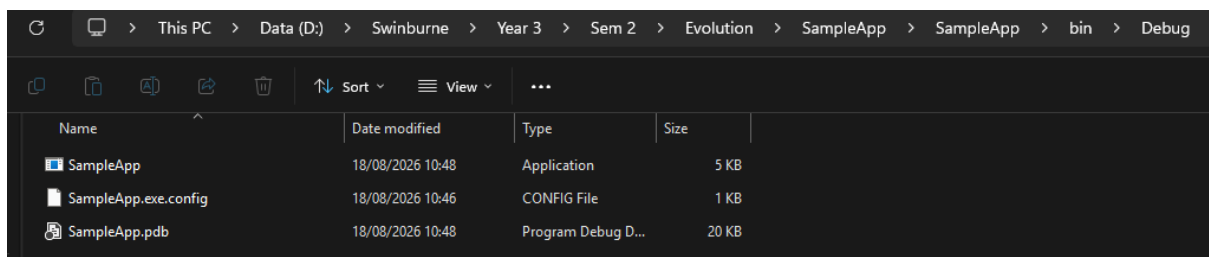
1. Code for SampleApp



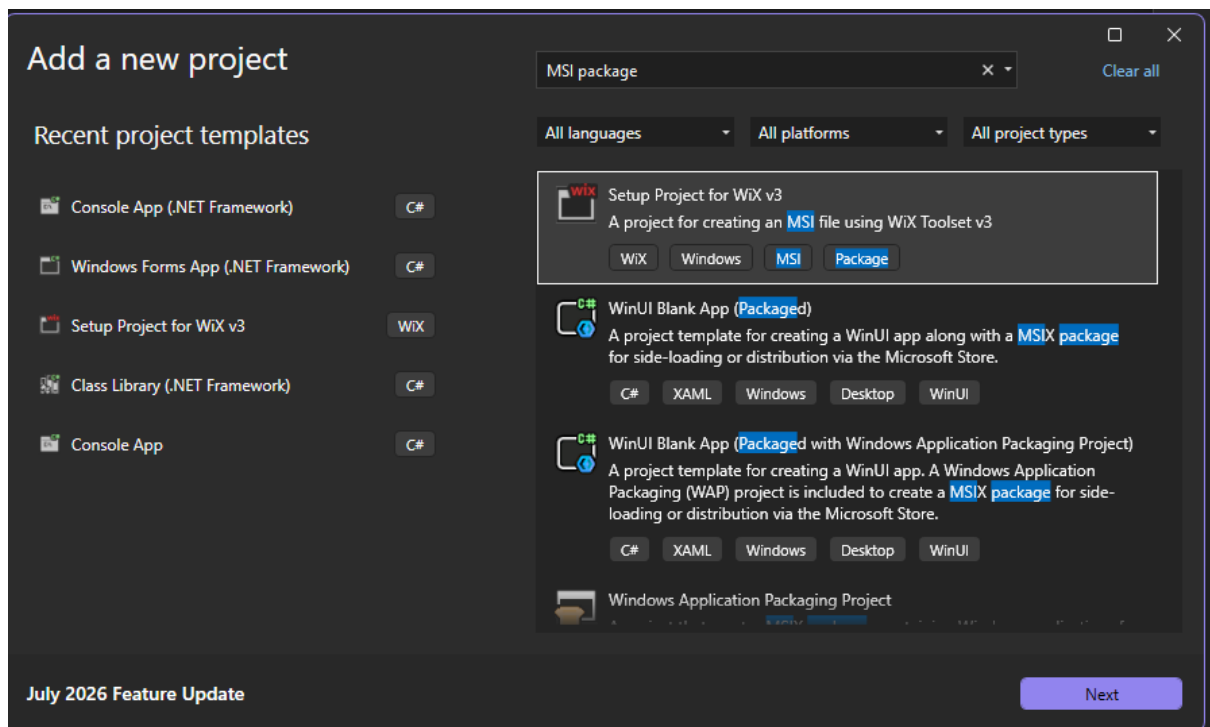
2. Building SampleApp



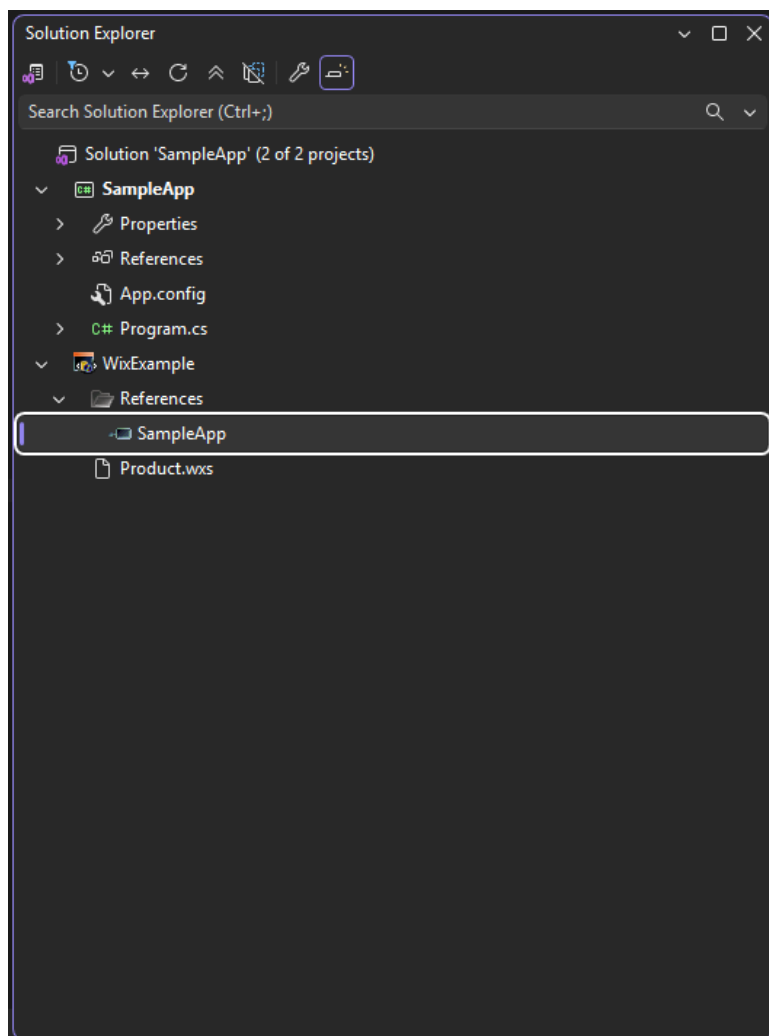
3. Folder consisting of SampleApp.exe



4. Setting up for SampleAppSetup



5. Add reference



6. SampleAppSetup (Product.wxs)

```
Product.wxs  X Program.cs
1  <?xml version="1.0" encoding="UTF-8"?>
2  <Wix xmlns="http://schemas.microsoft.com/wix/2006/wi">
3    <Product Id="*" Name="WixExample" Language="1033" Version="1.0.0.0" Manufacturer="SwinburneStudent" UpgradeCode="51d42271-27b9-43ef-b752-5fa542c6f4ce">
4      <Package InstallerVersion="200" Compressed="yes" InstallScope="perMachine" />
5
6      <MajorUpgrade DowngradeErrorMessage="A newer version of [ProductName] is already installed." />
7      <MediaTemplate EmbedCab="yes" />
8
9      <Feature Id="ProductFeature" Title="WixExample" Level="1">
10        <ComponentGroupRef Id="ProductComponents" />
11      </Feature>
12    </Product>
13
14    <Fragment>
15      <Directory Id="TARGETDIR" Name="SourceDir">
16        <Directory Id="ProgramFilesFolder">
17          <Directory Id="INSTALLFOLDER" Name="WixExample" />
18        </Directory>
19      </Directory>
20    </Fragment>
21
22    <Fragment>
23      <ComponentGroup Id="ProductComponents" Directory="INSTALLFOLDER">
24        <Component Id="ProductComponent">
25          <File Id="SampleAppEXE" Source="$(var.SampleApp.TargetPath)" KeyPath="yes" />
26        </Component>
27      </ComponentGroup>
28    </Fragment>
29  </Wix>
```

7. Build setup outcome

Output

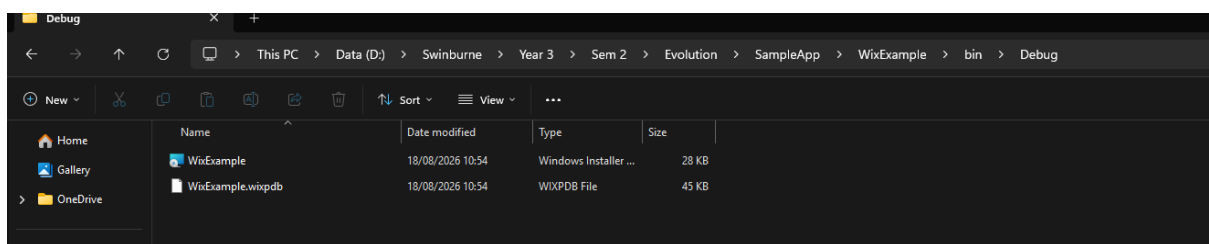
Show output from: Build

```
Build started at 11:03...
1>----- Build started: Project: WixExample, Configuration: Debug x86 -----
2> C:\Program Files (x86)\Wix Toolset v3.11\bin\wixc.exe -out "D:\Swinburne\Year 3\Sem 2\Evolution\SampleApp\WixExample\bin\Debug\WixExample.wix" -pout "D:\Swinburne\Year 3\Sem 2\Evolution\SampleApp\WixExample\bin\Debug\WixExample.wixpdb" -cultures:null -cont
3> WixExample -> D:\Swinburne\Year 3\Sem 2\Evolution\SampleApp\WixExample\bin\Debug\WixExample.wix
4> ===== Build: 1 succeeded, 0 failed, 1 up-to-date, 0 skipped =====
5> ===== Build completed at 11:03 and took 00,543 seconds =====
```

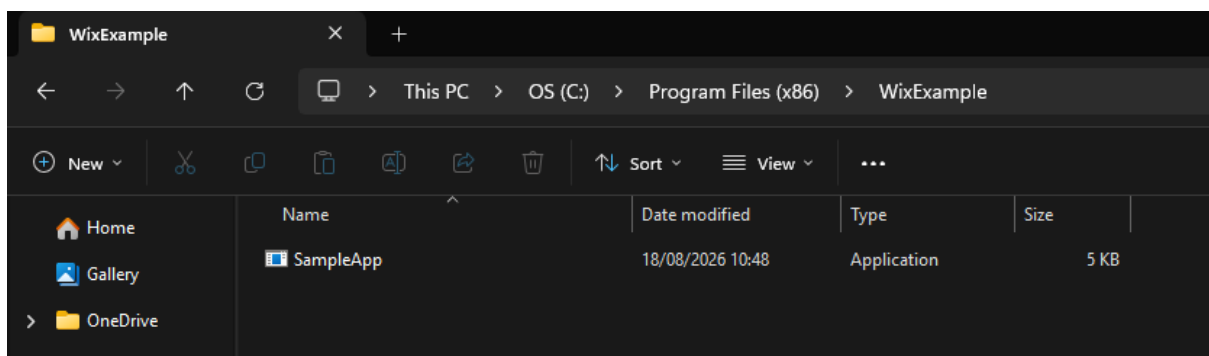
Show output from: Build

```
Build started at 11:04...
1>----- Build started: Project: WixExample, Configuration: Debug x86 -----
2> ===== Build: 1 succeeded, 0 failed, 1 up-to-date, 0 skipped =====
3> ===== Build completed at 11:04 and took 00,123 seconds =====
```

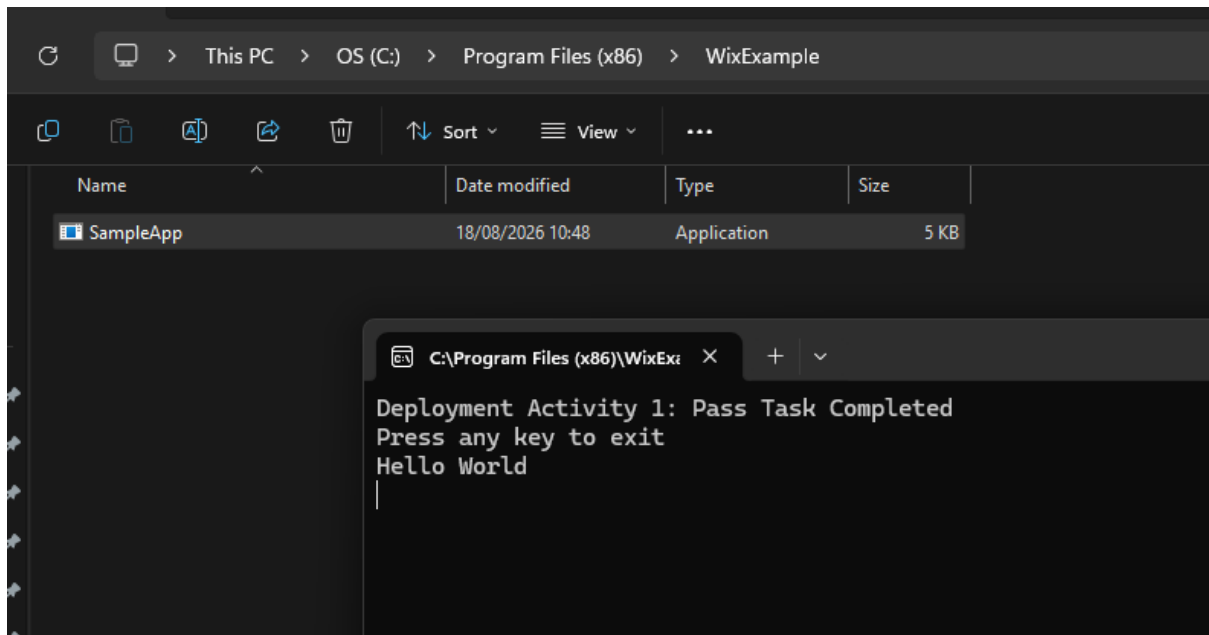
8. Run the window installer



9. File in C Drive, Program files



10. Output



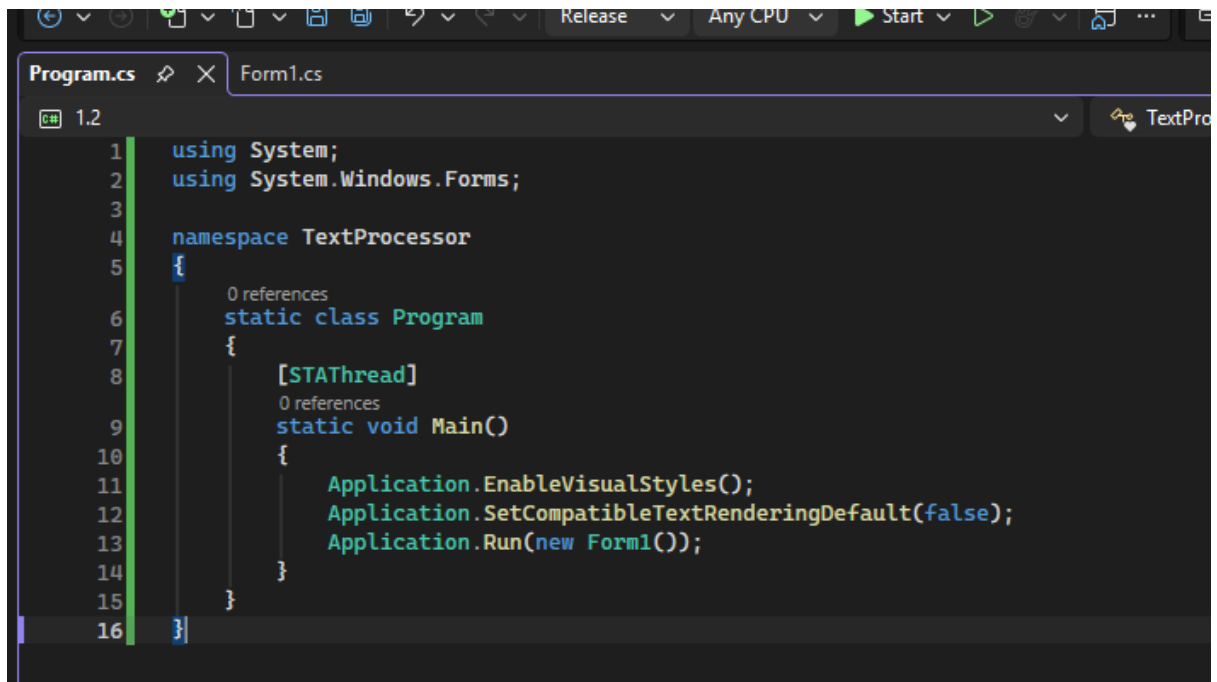
#instalation

1. go to SampleApp folder and there will be WixExample folder go in go to bin/release/ and then find the Window Installer or WixExample
2. click on it and wait for the download
3. go to C drive
4. find program files and then find WixExample
5. Run the Application

1.2 Simple Text Processor

For Task 1.2 I made a simple text processor where the user can type in the words or sentences, they want and simply press the count words button and the software will count how many words has been typed into the text box.

1. Code (Program.cs)



```
1 using System;
2 using System.Windows.Forms;
3
4 namespace TextProcessor
5 {
6     0 references
7     static class Program
8     {
9         [STAThread]
10        0 references
11        static void Main()
12        {
13            Application.EnableVisualStyles();
14            Application.SetCompatibleTextRenderingDefault(false);
15            Application.Run(new Form1());
16        }
17    }
18 }
```

2. Code (Form1.cs)

```
Program.cs  Form1.cs  X
1.2
1  using System;
2  using System.Windows.Forms;
3
4  namespace TextProcessor
5  {
6      2 references
7      public class Form1 : Form
8      {
9          private TextBox inputBox;
10         private Label resultsLabel;
11
12         1 reference
13         public Form1()
14         {
15             Text = "Word Counter";
16             Width = 400;
17             Height = 300;
18
19             // Text input
20             inputBox = new TextBox();
21             inputBox.Multiline = true;
22             inputBox.Dock = DockStyle.Top;
23             inputBox.Height = 150;
24             inputBox.Text = "Paste your text here...";
25             Controls.Add(inputBox);
26
27             // Button to count
28             Button countBtn = new Button();
29             countBtn.Text = "Count Words";
30             countBtn.Width = 100;
31             countBtn.Top = 160;
32             countBtn.Click += CountBtn_Click;
33             Controls.Add(countBtn);
34
35             // Results label
36             resultsLabel = new Label();
37             resultsLabel.Top = 200;
38             resultsLabel.Left = 10;
39             resultsLabel.Width = 360;
40             resultsLabel.Height = 50;
41             resultsLabel.BorderStyle = BorderStyle.FixedSingle;
42             resultsLabel.Font = new System.Drawing.Font("Arial", 14);
43             Controls.Add(resultsLabel);
44         }
45     }
46 }
```

```
1 reference
44 private void CountBtn_Click(object sender, EventArgs e)
45 {
46     if (string.IsNullOrEmpty(inputBox.Text) || inputBox.Text == "Paste your text here...")
47     {
48         MessageBox.Show("Enter some text first!");
49         return;
50     }
51
52     int wordCount = inputBox.Text.Split(new[] { ' ', '\n', '\t' }, StringSplitOptions.RemoveEmptyEntries).Length;
53     resultsLabel.Text = "Word Count: " + wordCount;
54 }
55 }
56 }
```

3. Build

Output

Show output from: Build

```
Build started at 12:00...
1>----- Build started: Project: 1.2, Configuration: Release Any CPU -----
1> 1.2 -> D:\Swinburne\Year 3\Sem 2\Evolution\1.2\1.2\1.2\bin\Release\1.2.exe
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 12:00 and took 00,335 seconds =====
```

File Explorer view of the build output directory: D:\Swinburne\Year 3\Sem 2\Evolution\1.2\1.2\1.2\bin\Release

Name	Date modified	Type	Size
1.2	20/08/2026 07:38	Application	8 KB
1.2.exe.config	19/08/2026 11:54	CONFIG File	1 KB
1.2.pdb	20/08/2026 07:38	Program Debug D...	30 KB

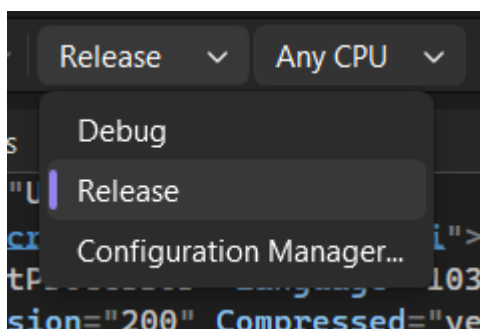
4. 1.2 setup (Product.wxs, add 1.2 as reference)

Product.wxs

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <Wix xmlns="http://schemas.microsoft.com/wix/2006/wi">
3   <Product Id="*" Name="TextProcessor" Language="1033" Version="1.0.0.0" Manufacturer="SwinburneStudent" UpgradeCode="c82ac965-b43a-4c02-9439-8626cdb4ee4c">
4     <Package InstallerVersion="200" Compressed="yes" InstallScope="perMachine" />
5
6     <MajorUpgrade DowngradeErrorMessage="A newer version of [ProductName] is already installed." />
7     <MediaTemplate EmbedCab="yes" />
8
9     <Feature Id="ProductFeature" Title="TextProcessor" Level="1">
10       <ComponentGroupRef Id="ProductComponents" />
11     </Feature>
12   </Product>
13
14   <Fragment>
15     <Directory Id="TARGETDIR" Name="SourceDir">
16       <Directory Id="ProgramFilesFolder">
17         <Directory Id="INSTALLFOLDER" Name="TextProcessor" />
18       </Directory>
19     </Directory>
20   </Fragment>
21
22   <Fragment>
23     <ComponentGroup Id="ProductComponents" Directory="INSTALLFOLDER">
24       <Component Id="ProductComponent">
25         <File Id="TextProcessorEXE" Source="$(var.1.2.TargetPath)" KeyPath="yes" />
26       </Component>
27     </ComponentGroup>
28   </Fragment>
29 </Wix>
```

Solution Explorer: Solution '1.2' (2 of 2 projects)

- 1.2
 - Properties
 - References
 - App.config
 - Form1.cs
 - Form1
 - Form1.resx
 - Program.cs
 - 1.2 Setup
 - References
 - 1.2
 - Product.wxs

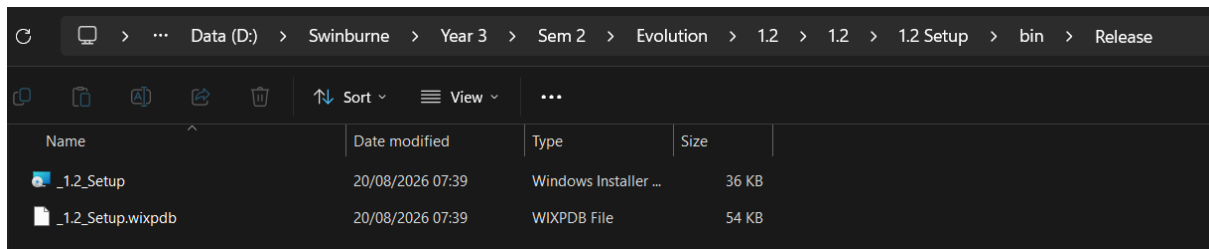


Output

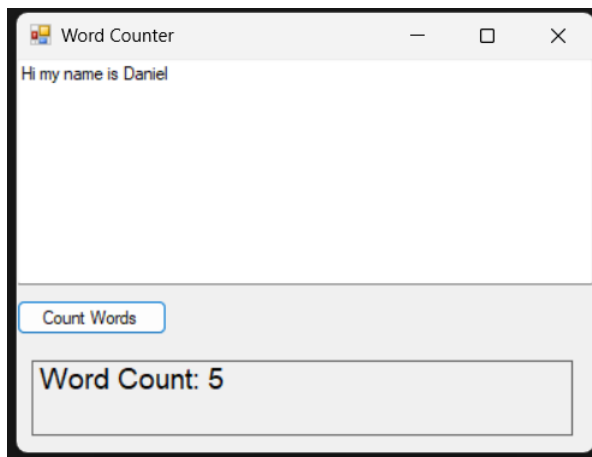
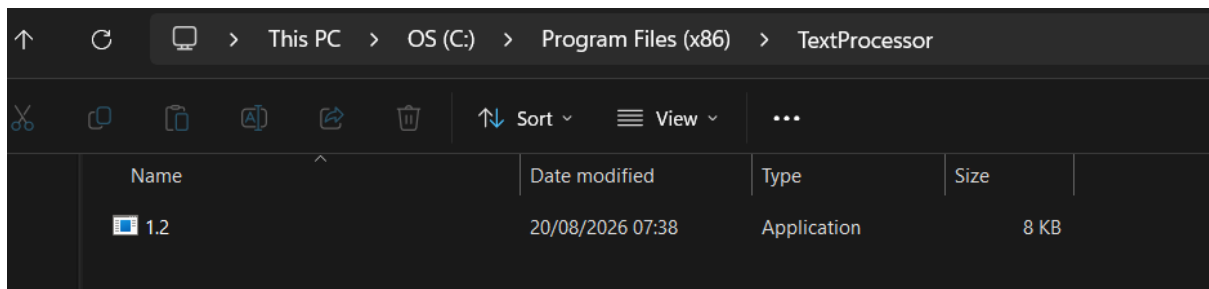
Show output from: Build

```
Build started at 07:39...
1>----- Build started: Project: 1.2 Setup, Configuration: Release x86 -----
1> C:\Program Files (x86)\WIX Toolset v3.11\bin\candle.exe -d:"DevFndr=C:\Program Files\Microsoft Visual Studio\18\Professional\Common7\IDE\\" -d:"SolutionDir=D:\Swinburne\Year 3\Sem 2\Evolution\1.2\1.2\\" -d:"SolutionExt=.slnx" -d:"
1> C:\Program Files (x86)\WIX Toolset v3.11\bin\light.exe -out "D:\Swinburne\Year 3\Sem 2\Evolution\1.2\1.2\1.2 Setup\bin\Release\1.2_Setup.msi" -pdbout "D:\Swinburne\Year 3\Sem 2\Evolution\1.2\1.2\1.2 Setup\bin\Release\1.2_Setup.msi"
1> 1.2 Setup -> D:\Swinburne\Year 3\Sem 2\Evolution\1.2\1.2\1.2 Setup\bin\Release\1.2_Setup.msi
===== Build: 1 succeeded, 0 failed, 1 up-to-date, 0 skipped =====
===== Build completed at 07:39 and took 04,538 seconds =====
```

5. Windows installer



6. App (file location in C drive, and app output)



1.3 Adding DLLs

for 1.3 I created 2 libraries and add into the code from Task 1.2 (TextProcessorLibrary and TextProcessorsUtils)

1. TextProcessorsLibrary

- Contains the WordCounter Class, which counts the number of words in the input text

```
Program.cs  Form1.cs  Form1.cs [Design]  Class1.cs  Class1.cs  TextProcessorLib.WordCounter
1  using System;
2
3  namespace TextProcessorLib
4  {
5      2 references | 0 changes | 0 authors, 0 changes
6      public class WordCounter
7      {
8          1 reference | 0 changes | 0 authors, 0 changes
9          public int CountWords(string text)
10         {
11             if (string.IsNullOrEmpty(text))
12                 return 0;
13             return text.Split(new[] { ' ', '\n', '\t' }, StringSplitOptions.RemoveEmptyEntries).Length;
14         }
15     }
```

Show output from: Build

```
Build started at 13:55...
1>----- Build started: Project: 1.2, Configuration: Release Any CPU -----
2> 1.2 -> D:\Swinsburne\Year 3\Sem 2\Evolution\1.3\1.2\bin\Release\1.2.exe
3>----- Build started: Project: 1.2 Setup, Configuration: Release x86 -----
4> C:\Program Files (x86)\MSI Toolset v3.11\bin\msiexec.exe -q "D:\Swinsburne\Year 3\Sem 2\Evolution\1.3\1.2 Setup\bin\Release\1.2_Setup.msi" -pout "D:\Swinsburne\Year 3\Sem 2\Evolution\1.3\1.2 Setup\bin\Release\1.2_Setup.wixpdb" -cultures:en -contentsfile "
5> C:\Program Files (x86)\MSI Toolset v3.11\bin\msiexec.exe -out "D:\Swinsburne\Year 3\Sem 2\Evolution\1.3\1.2 Setup\bin\Release\1.2_Setup.msi"
6> 1.2 Setup -> D:\Swinsburne\Year 3\Sem 2\Evolution\1.3\1.2 Setup\bin\Release\1.2_Setup.msi
7>----- Build: 2 succeeded, 0 failed, 2 up-to-date, 0 skipped -----
8>----- Build completed at 13:55 and took 00,398 seconds -----
```

2. TextProcessorsUtils

- Contains the TextValidator class, which validates that the user has entered valid text before counting.

```
TextProcessorUtils
1  using System;
2
3  namespace TextProcessorUtils
4  {
5      2 references | 0 changes | 0 authors, 0 changes
6      public class TextValidator
7      {
8          1 reference | 0 changes | 0 authors, 0 changes
9          public bool IsValidText(string text)
10         {
11             return !string.IsNullOrEmpty(text) && text != "Paste your text here...";
12         }
13     }
```

Show output from: Build

```
Build started at 13:56...
1>----- Build started: Project: 1.2 Setup, Configuration: Release x86 -----
2>----- Build: 1 succeeded, 0 failed, 3 up-to-date, 0 skipped -----
3>----- Build completed at 13:56 and took 00,404 seconds -----
```

3. Updated Form1.cs

```
Program.cs  Form1.cs  Form1.cs [Design]  Class1.cs

1.2
1  using System;
2  using System.Windows.Forms;
3  using TextProcessorLib;
4  using TextProcessorUtils;
5
6  namespace TextProcessor
7  {
8      2 references | 0 changes | 0 authors, 0 changes
9      public class Form1 : Form
10     {
11         private TextBox inputBox;
12         private Label resultsLabel;
13         private WordCounter wordCounter;
14         private TextValidator validator;
15
16         1 reference | 0 changes | 0 authors, 0 changes
17         public Form1()
18         {
19             wordCounter = new WordCounter();
20             validator = new TextValidator();
21             Text = "Word Counter";
22             Width = 400;
23             Height = 300;
24
25             // Text input
26             inputBox = new TextBox();
27             inputBox.Multiline = true;
28             inputBox.Dock = DockStyle.Top;
29             inputBox.Height = 150;
30             inputBox.Text = "Paste your text here...";
31             Controls.Add(inputBox);
32
33             // Button to count
34             Button countBtn = new Button();
35             countBtn.Text = "Count Words";
36             countBtn.Width = 100;
37             countBtn.Top = 160;
38             countBtn.Click += CountBtn_Click;
39             Controls.Add(countBtn);
40
41             // Results label
42             resultsLabel = new Label();
43             resultsLabel.Top = 200;
44             resultsLabel.Left = 10;
45             resultsLabel.Width = 360;
46             resultsLabel.Height = 50;
47             resultsLabel.BorderStyle = BorderStyle.FixedSingle;
48             resultsLabel.Font = new System.Drawing.Font("Arial", 14);
49             Controls.Add(resultsLabel);
50
51             1 reference | 0 changes | 0 authors, 0 changes
52             private void CountBtn_Click(object sender, EventArgs e)
53             {
54                 if (!validator.IsValidText(inputBox.Text))
55                 {
56                     MessageBox.Show("Enter some text first!");
57                     return;
58                 }
59
60                 int wordCount = wordCounter.CountWords(inputBox.Text);
61                 resultsLabel.Text = $"Word Count: {wordCount}";
62             }
63         }
64     }
65 }
```

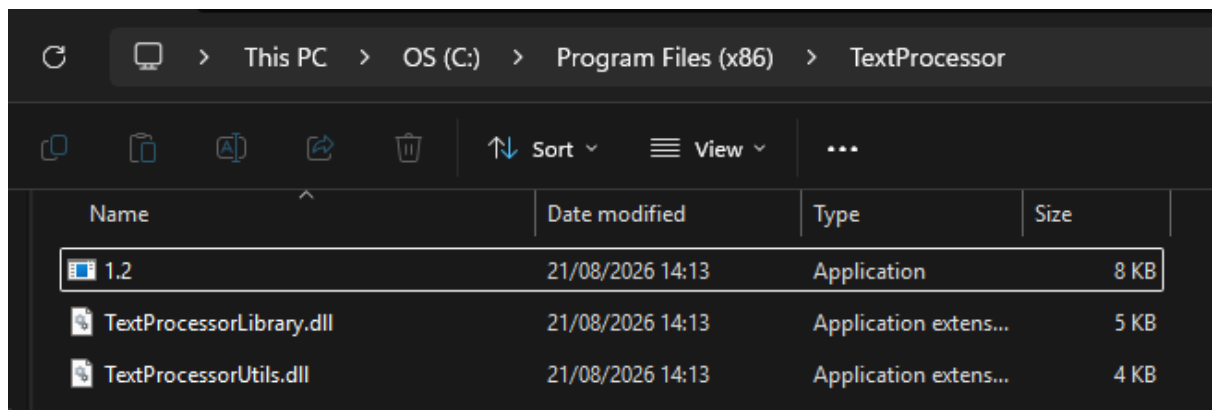
4. Setup

5. Output

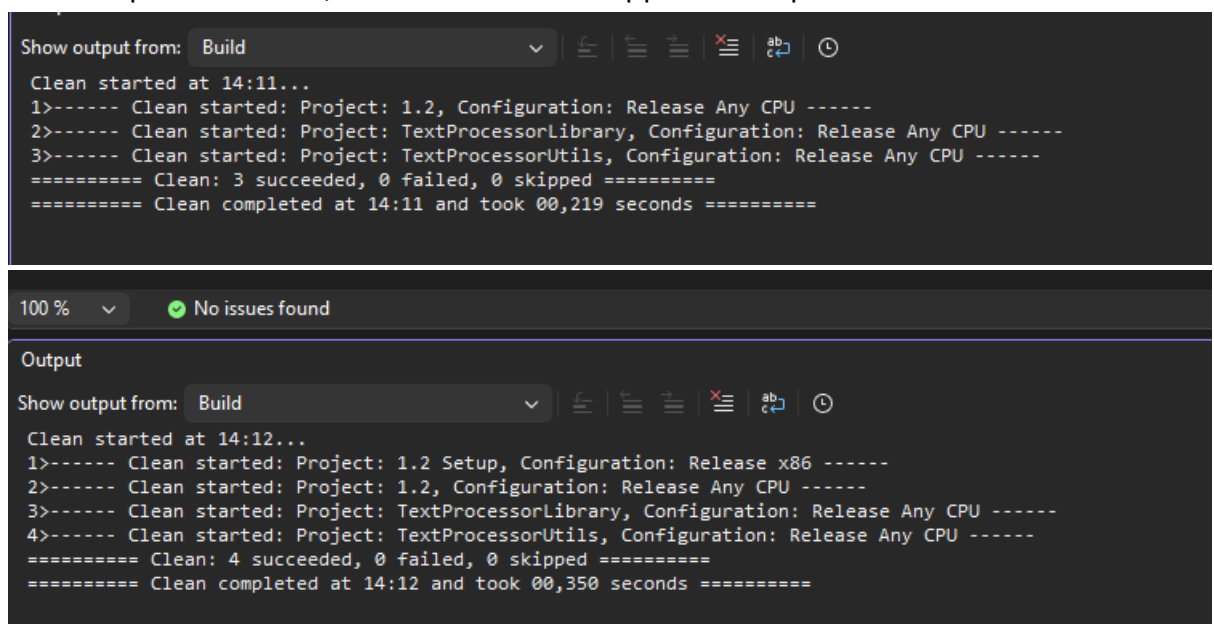
Download the application

Count Words

Word Count: 7



In this 1.3 I also clean before building it again to deletes all the old compiled files from the previous build; and also delete the app from the previous download



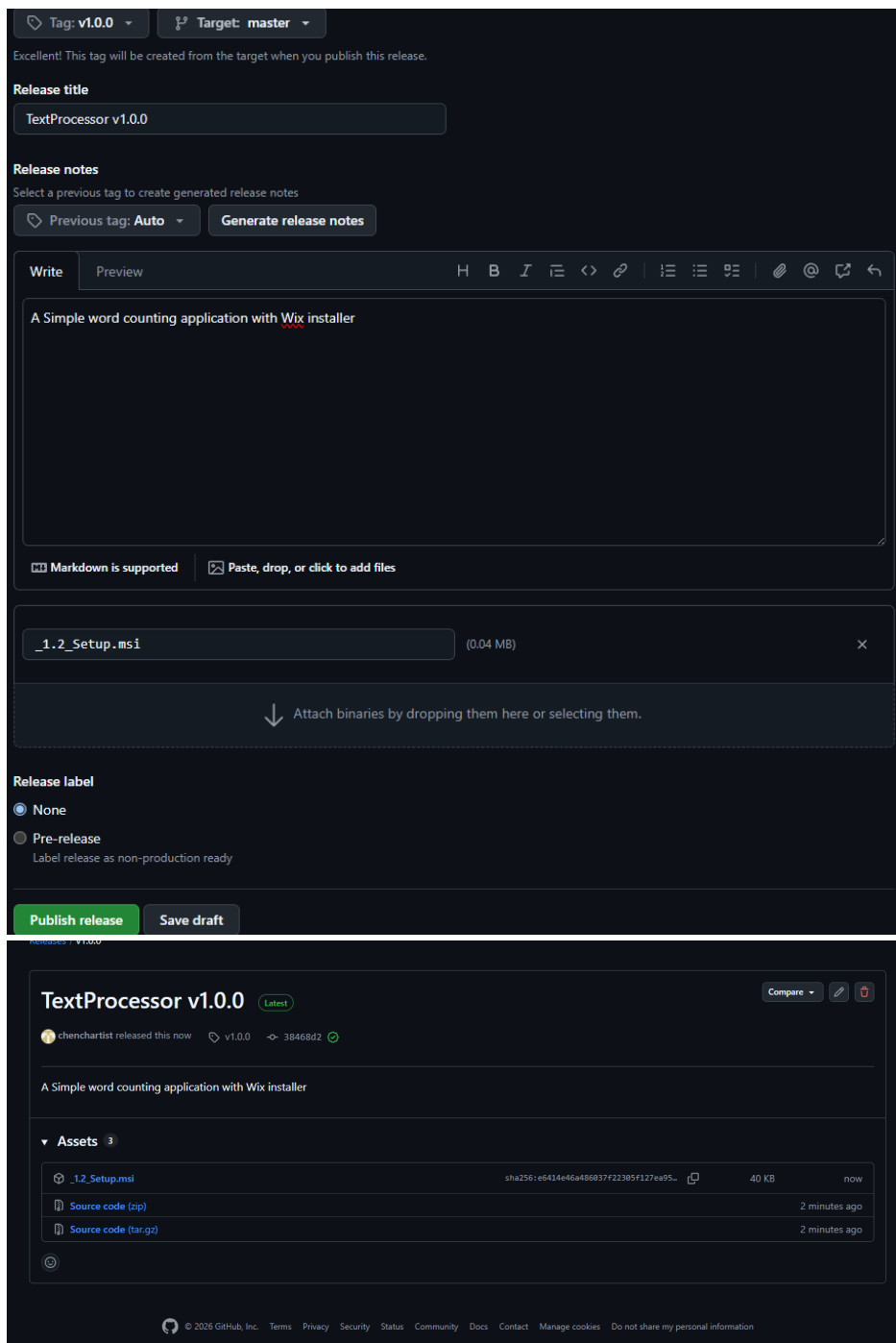
1.4 GitHub Release + WinGet

For 1.4 we will be releasing the code on GitHub and create a manifest on WinGet

Step by step

1. Create GitHub release

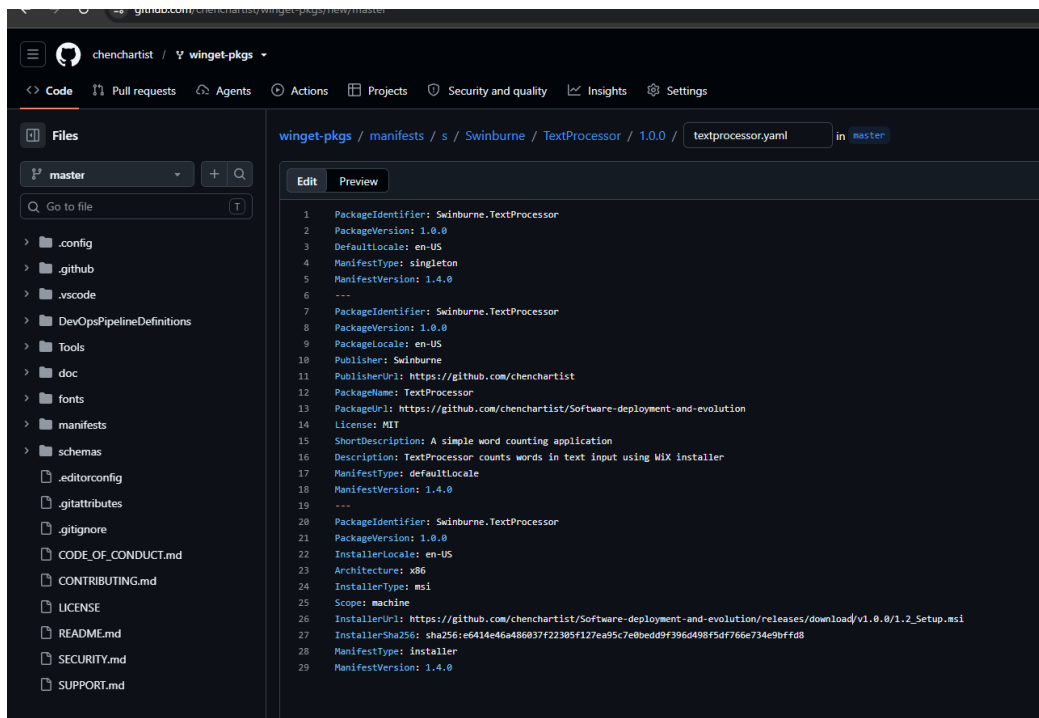
- Go to GitHub repository, on the right side click on release, click on create a new release, select your file, publish release



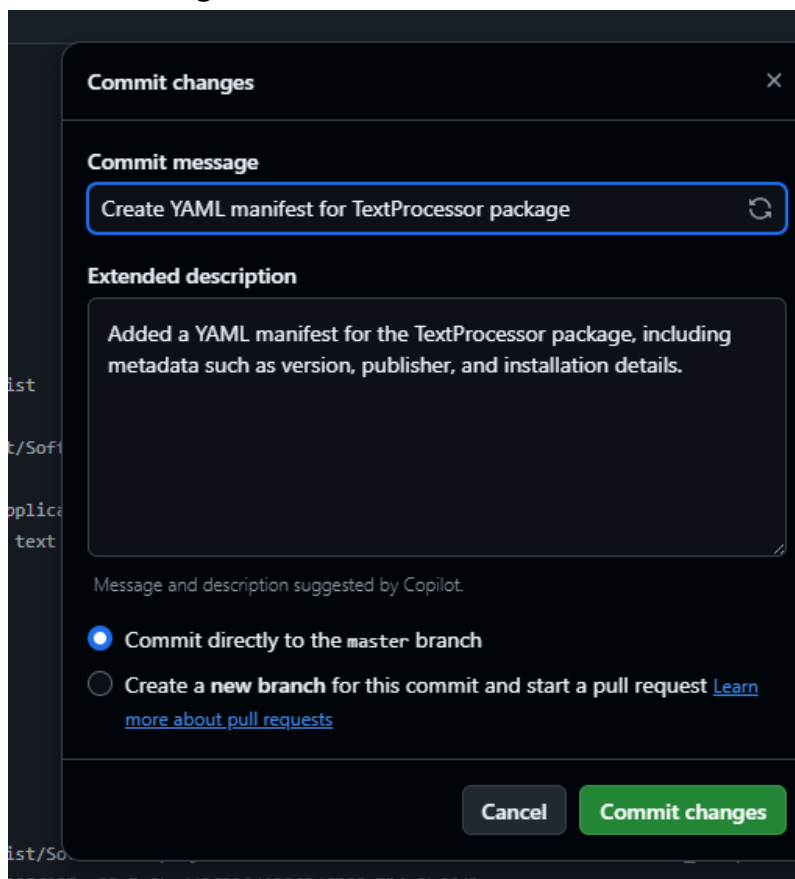
2. Get the SHA, keep it, will need for the manifest file

(sha256:e6414e46a486037f22305f127ea95c7e0bedd9f396d498f5df766e734e9bffd8)

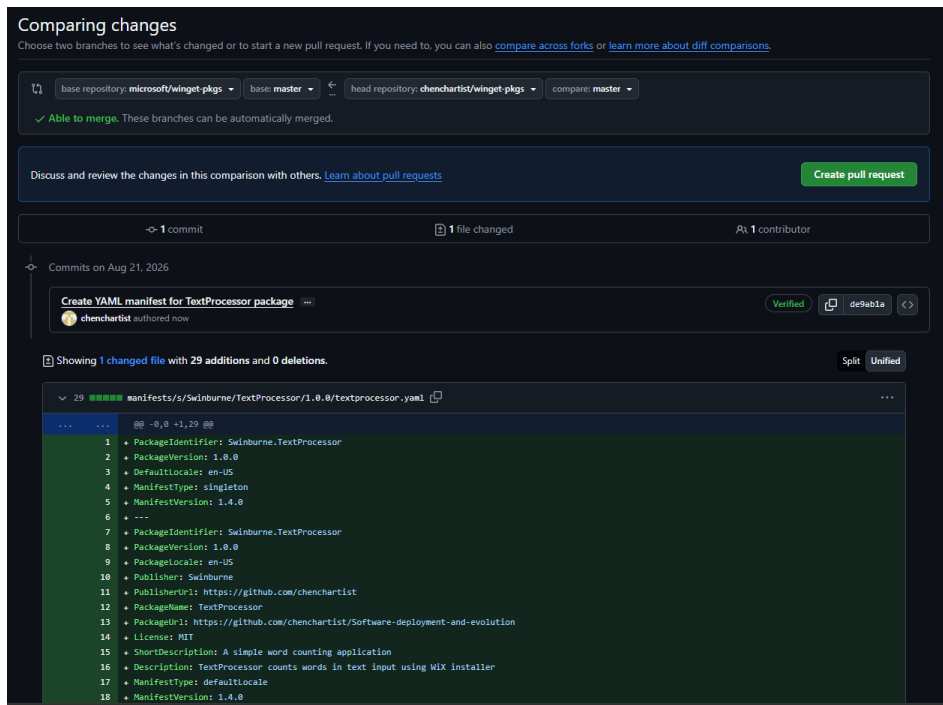
3. Create Manifest (the .yaml content), and put in the sha in InstallerSha256



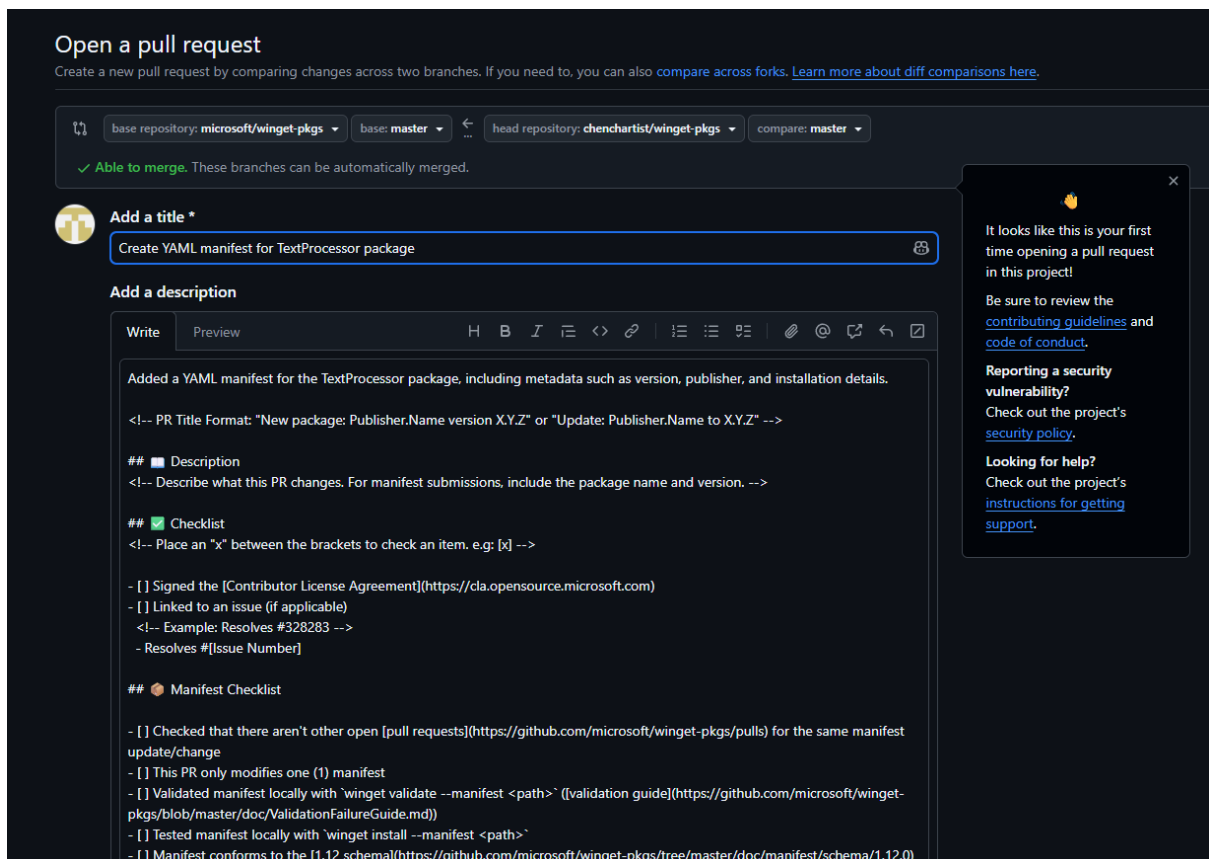
4. Go to “wingetpkgs-fork” and fork ur github [username]/winget-pkgs
5. Click “add-file” > “create new file”
6. Copy the Yaml content
7. Commit changes



8. Comparing changes



9. Create pull request



10. Create Title and write description, submit pull request to Microsoft



Add a title *

Add TextProcessor v1.0.0 Windows package

Add a description

WritePreview

H B I           

Description

Added a YAML manifest for the `TextProcessor` package, including metadata such as version, publisher, and installation details.

Package: `TextProcessor v1.0.0`
Publisher: `Swinburne`
Installer: `Windows MSI package (1.2_Setup.msi)`

Manifest Checklist

- [x] This PR only modifies one (1) manifest
- [x] Validated manifest locally with `winget validate --manifest <path>`
- [x] Tested manifest locally with `winget install --manifest <path>`
- [x] Manifest conforms to the schema

Related Issues

Adds Windows Package Manager support for `TextProcessor` word counting application.

 Markdown is supported

 Paste, drop, or click to add files

☒ Allow edits by maintainers 

Create pull request 

 Remember, contributions to this repository should follow its [contributing guidelines](#), [security policy](#), and [code of conduct](#).

11. Created

Add TextProcessor v1.0.0 Windows package #422078 

 Open

`chenchartist` wants to merge 1 commit into `microsoft:master` from `chenchartist:master` 

 Conversation 0

 Commits 1

 Checks 0

 Files changed 1



`chenchartist` commented *now* 

Description

Added a YAML manifest for the `TextProcessor` package, including metadata such as version, publisher, and installation details.

Package: `TextProcessor v1.0.0`
Publisher: `Swinburne`
Installer: `Windows MSI package (1.2_Setup.msi)`

Manifest Checklist

- ☒ This PR only modifies one (1) manifest
- ☒ Validated manifest locally with `winget validate --manifest <path>`
- ☒ Tested manifest locally with `winget install --manifest <path>`
- ☒ Manifest conforms to the schema

Related Issues

Adds Windows Package Manager support for `TextProcessor` word counting application.

Reviewers

No reviews

Still in progress

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merged issues.

None yet

Notifications

